

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する

Notebook handout

Source notebook: [Transolver_Learn/japanese_notebooks/02_physics_attention_transolver_ja.ipynb](#)

b

このPDFはNotebook本文、標準出力、matplotlib図を含む確認用資料です。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 1

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する

Notebook 1では、標準Transformerをそのまま大規模メッシュに使うと $O(N^2)$ のAttentionが重くなることを見ました。

このNotebookでは、Transolverの中心である Physics-Attention をNumPyだけで実装します。目的は高性能な学習コードを書くことではなく、各ステップのshapeと意味を理解することです。

このNotebookで学ぶこと

1. 点群 N 個を少数のslice M 個へsoftに割り当てる
2. 点特徴をslice tokenへ集約する
3. slice token間でAttentionする
4. 更新されたtokenを各点へ戻す
5. この処理が標準Attentionより軽くなる理由をshapeで理解する

用語メモ

- feature / channel: 各点を持つ数値特徴。座標、法線、圧力、速度、埋め込みベクトルなど。
- embedding: 生の入力をニューラルネットが扱いやすいベクトルへ変換したもの。
- soft assignment: 1つの点を1つのグループに固定するのではなく、複数グループへの所属度を確率的な重みで表すこと。
- bottleneck: 情報を一度少数の表現へ圧縮する箇所。計算量を減らす一方、重要な情報を落とさない設計が必要。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 2

1. Toy Dataで考える理由

本来のTransolverは、CFD surface meshやPDEデータのような大きなデータに使います。ただし、最初から数十万点のデータを扱っていると、shapeや計算の意味が見えにくくなります。

そこでここでは、以下の小さなtoy problemを使います。

```
```text
N = 1000 点数
M = 8 slice / physics token数
C = 32 各点の特徴次元
```
```

実データではNはもっと大きく、Mは32や64などが使われます。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 3

<!-- codex-code-explanation -->

コード解説: Toy Dataの設定

このセルでは、Physics-AttentionをNumPyだけで追うための小さな問題を作ります。N はメッシュ点数、M はphysics sliceまたはtokenの数、C は各点を持つ特徴量の次元です。実際のTransolverでは、x は座標、法線、境界条件、途中層の特徴などをニューラルネットに変換した表現に対応します。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Code 1

```
# --- NumPyでPhysics-Attentionの計算を手で追うための準備です。 ---
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# --- Attentionで使うsoftmaxをNumPyで定義します。 ---
def softmax(x, axis=-1):
    e_x = np.exp(x - np.max(x, axis=axis, keepdims=True))
    return e_x / e_x.sum(axis=axis, keepdims=True)

# --- Toy問題のサイズを決めます。Nは点数、Mはslice数、Cは特徴次元です。 ---
N = 1000 # mesh points
M = 8   # physics slices / tokens
C = 32  # feature channels

print("Problem setup")
print(f" N: mesh points   = {N}")
print(f" M: physics slices = {M}")
print(f" C: feature channels = {C}")

# xは、各点がC次元の特徴を持つ行列です。
# 実際には座標・法線・物理量などをニューラルネットで埋め込んだ表現に相当します。
# --- 各点がC次元特徴を持つ、仮想的な点群特徴を作ります。 ---
x = np.random.rand(N, C)
print("Input x shape:", x.shape)
```

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Code 1 output

Problem setup

N: mesh points = 1000

M: physics slices = 8

C: feature channels = 32

Input x shape: (1000, 32)

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 5

2. Step 1: Slice

最初に、各点を M 個のsliceへ割り当てます。

ここで重要なのは、K-meansのように「点*i*はslice 3だけ」と固定するのではなく、各点が複数sliceに重みを持つことです。

```
```text
x: (N, C)
W_project: (C, M)
slice_logits: (N, M)
slice_weights: (N, M)
```
```

`slice_weights[i, j]` は「点 *i* が slice *j* にどの程度属するか」を表します。各点についてslice方向にsoftmaxを取るので、各行の合計は1です。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 6

<!-- codex-code-explanation -->

コード解説: Step 1 Slice

このセルでは、各点を M 個のsliceへsoftに割り当てます。slice_logits = x @ W_project は「この点はこのsliceに属しやすいか」を表すスコアで、softmax により各点について合計1の重みに変換します。hardなクラスタリングではなくsoft assignmentにすることで、1つの点が複数の物理構造にまたがって寄与できます。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Code 2

```
# --- Step 1: 各点をどのsliceへ割り当てるかを計算します。 ---
# C次元特徴からM個のslice重みを作るための行列。
# 実際のモデルでは学習で更新されます。
W_project = np.random.rand(C, M)

# --- 点特徴をslice方向へ射影し、割り当て前のスコアを作ります。 ---
slice_logits = x @ W_project      # (N, C) @ (C, M) -> (N, M)
# --- softmaxにより、各点についてM個のslice重みの合計を1にします。 ---
slice_weights = softmax(slice_logits, axis=1)

print("slice_logits shape: ", slice_logits.shape)
print("slice_weights shape:", slice_weights.shape)
print("row sums first 5: ", np.round(slice_weights[:5].sum(axis=1), 6))

# --- 点ごとのslice割り当てをヒートマップで確認します。 ---
plt.figure(figsize=(10, 4))
plt.imshow(slice_weights.T, aspect="auto", cmap="viridis")
plt.colorbar(label="slice weight")
plt.xlabel("mesh point index")
plt.ylabel("slice index")
plt.title("Step 1: Soft assignment from points to slices")
plt.tight_layout()
plt.show()
```

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Code 2 output

slice_logits shape: (1000, 8)

slice_weights shape: (1000, 8)

row sums first 5: [1. 1. 1. 1. 1.]

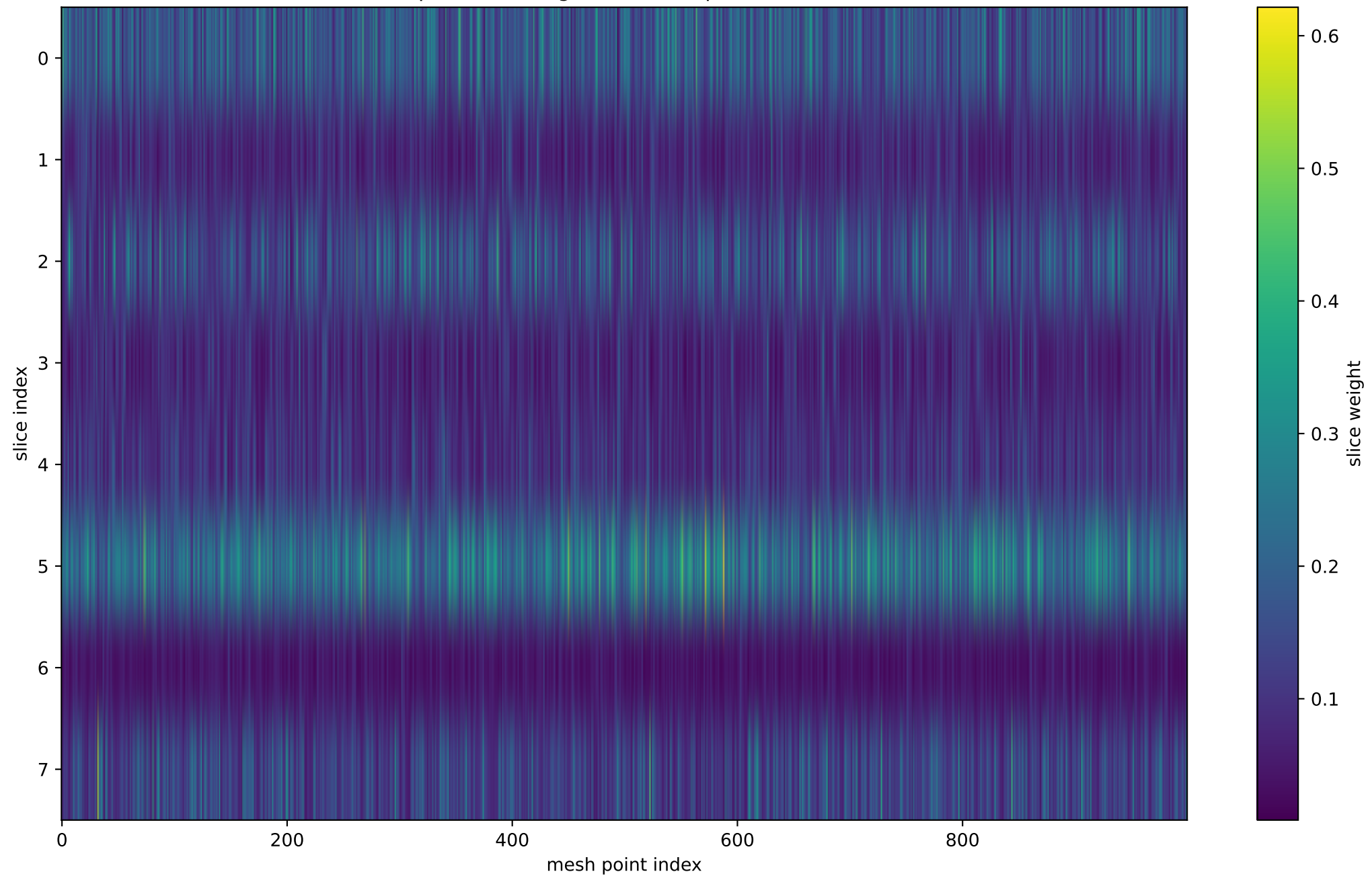
02_physics_attention_transolver_ja.ipynb:cell7:23: UserWarning: FigureCanvasAgg is non-interactive, and thus c

annot be s

hown

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Code 2 - Figure 1

Step 1: Soft assignment from points to slices



Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 8

上のヒートマップでは、横軸が点、縦軸がsliceです。明るいほど、その点がそのsliceに強く割り当てられていることを示します。

重要な直感

sliceは単なる空間分割とは限りません。学習が進むと、あるsliceが「前縁付近の高圧領域」「後流に
関係する領域」「剥離に関する状態」のような、物理的に意味のある代表状態を担う可能性があります。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 9

3. Step 2: Aggregate

次に、点ごとの特徴 x をsliceごとに集約し、 M 個のtokenを作ります。

```
```text
slice_weights.T: (M, N)
x: (N, C)
z: (M, C)
```
```

$z[j]$ はslice j を代表する特徴ベクトルです。点を重み付き平均して作ります。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 10

<!-- codex-code-explanation -->

コード解説: Step 2 Aggregate

このセルでは、点ごとの特徴 x をslice重みで加重平均し、 M 個のphysics token z に圧縮します。ここで N 点の情報が M tokenに集約されます。Transolverでは、このtokenが流れ場の代表的な構造、例えば前縁付近、後流、圧力勾配が強い領域などを表すように学習されることを期待します。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Code 3

--- Step 2: 点特徴をslice重みで加重平均し、physics tokenへ集約します。 ---

```
z_numerator = slice_weights.T @ x      # (M, N) @ (N, C) -> (M, C)
```

--- 各sliceに集まった重みの総量で割り、平均として正規化します。 ---

```
z_denominator = slice_weights.sum(axis=0)  # (M,)
```

```
z = z_numerator / (z_denominator.reshape(M, 1) + 1e-6)
```

```
print("z_numerator shape: ", z_numerator.shape)
```

```
print("z_denominator shape:", z_denominator.reshape(M, 1).shape)
```

```
print("physics token z:  ", z.shape)
```

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Code 3 output

z_numerator shape: (8, 32)

z_denominator shape: (8, 1)

physics token z: (8, 32)

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 12

ここで、1000点の情報が8個のtokenに圧縮されました。もちろん、ただ平均するだけでは情報が落ちます。Transolverでは、この割り当てと後続の変換が学習されるため、タスクに必要な情報を残すように調整されます。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 13

4. Step 3: Token Attention

標準Transformerなら $N \times N$ のAttentionを計算します。ここでは、点ではなく M 個のtoken間でAttentionします。

```
```text
Q_tokens: (M, C)
K_tokens: (M, C)
V_tokens: (M, C)
attention: (M, M)
```
```

M は N よりずっと小さいので、ここが計算量削減の中心になります。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 14

<!-- codex-code-explanation -->

コード解説: Step 3 Token Attention

このセルでは、集約されたphysics tokenどうしに標準的なAttentionをかけます。比較対象が全点 N ではなくslice数 M なので、Attention行列は $M \times M$ で済みます。ここがTransolverの中核で、細かい点群上で直接全比較するのではなく、物理構造の代表token間で相互作用を学習します。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Code 4

```
# --- Step 3: slice token同士でAttentionするため、Q/K/Vの線形変換を用意します。 ---
```

```
W_q = np.random.rand(C, C)
```

```
W_k = np.random.rand(C, C)
```

```
W_v = np.random.rand(C, C)
```

```
# --- 各physics tokenをQuery、Key、Value表現へ変換します。 ---
```

```
Q_tokens = z @ W_q
```

```
K_tokens = z @ W_k
```

```
V_tokens = z @ W_v
```

```
# --- token同士の類似度を計算します。ここではM x Mの小さいAttention行列です。 ---
```

```
scores_tokens = (Q_tokens @ K_tokens.T) / np.sqrt(C) # (M, M)
```

```
attn_weights_tokens = softmax(scores_tokens, axis=1)
```

```
z_prime = attn_weights_tokens @ V_tokens      # (M, C)
```

```
print("token attention shape:", attn_weights_tokens.shape)
```

```
print("updated token shape: ", z_prime.shape)
```

```
# --- physics token間のAttention重みを表示します。 ---
```

```
plt.figure(figsize=(4, 3))
```

```
plt.imshow(attn_weights_tokens, cmap="viridis")
```

```
plt.colorbar(label="attention weight")
```

```
plt.xlabel("Key slice")
```

```
plt.ylabel("Query slice")
```

```
plt.title("Step 3: Attention among physics tokens")
```

```
plt.tight_layout()
```

```
plt.show()
```

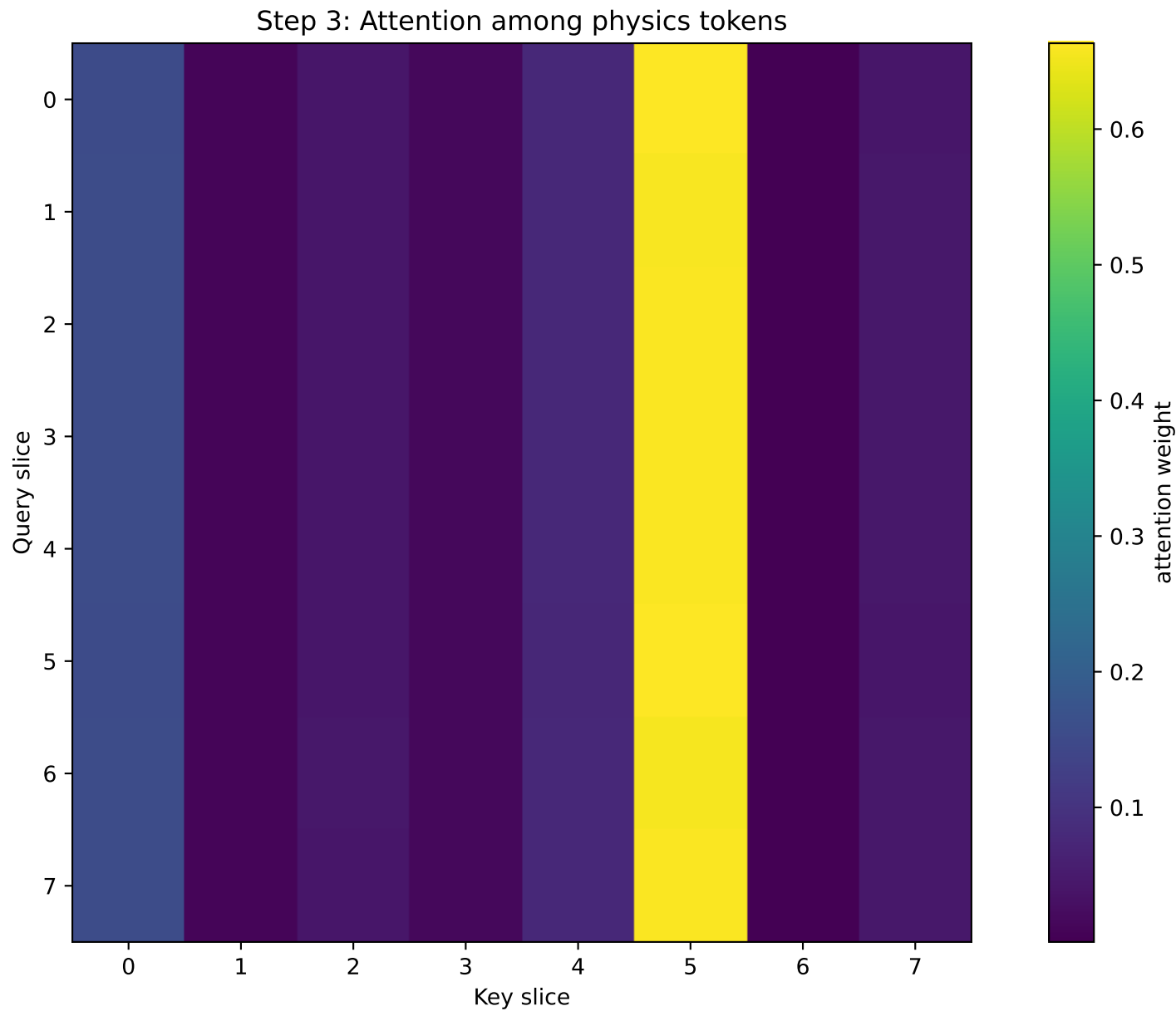
Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Code 4 output

token attention shape: (8, 8)

updated token shape: (8, 32)

02_physics_attention_transolver_ja.ipynb:cell15:27: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Code 4 - Figure 1



Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 16

この $M \times M$ 行列は、slice tokenどうしの相互作用を表します。例えば、あるsliceが「前方の圧力構造」、別のsliceが「後流構造」を表しているなら、それらの関係をここで学習できます。

注意: このNotebookではランダム行列を使っているため、図のパターン自体に物理的意味はありません。意味が出るのは実データで学習した後です。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 17

5. Step 4: Deslice / Broadcast

最後に、更新されたtoken z_{prime} を各点へ戻します。戻すときにも、Step 1で作った slice_weights を使います。

```
```text
slice_weights: (N, M)
z_prime: (M, C)
x_prime: (N, C)
```
```

出力shapeは入力 x と同じ (N, C) です。これにより、Physics-Attention層を何層も重ねられます。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 18

<!-- codex-code-explanation -->

コード解説: Step 4 Deslice

このセルでは、更新されたphysics token z_{prime} を元の点群へ戻します。slice_weights @ z_{prime} によって、各点は自分が属するsliceの情報を重み付きで受け取ります。出力shapeが入力と同じ $N \times C$ に戻るため、この処理を通常のニューラルネット層のように何層も積み重ねられます。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Code 5

--- Step 4: 更新されたtoken情報を、元の各点へ戻します。 ---

```
x_prime = slice_weights @ z_prime
```

```
print("slice_weights shape:", slice_weights.shape)
```

```
print("z_prime shape:  ", z_prime.shape)
```

```
print("x_prime shape:  ", x_prime.shape)
```

```
print("input x shape:  ", x.shape)
```

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Code 5 output

slice_weights shape: (1000, 8)

z_prime shape: (8, 32)

x_prime shape: (1000, 32)

input x shape: (1000, 32)

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 20

入力と出力のshapeが同じなので、次のTransolver層へそのまま渡せます。これは通常のTransformer blockと同じく、層を積み重ねやすい設計です。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 21

6. 4ステップを1つのクラスにまとめる

ここまでの処理を、簡単な PhysicsAttention クラスにまとめます。実際のPyTorch実装はもっと複雑ですが、shapeと流れは同じです。

ここでの対応は次の通りです。

```
```text
Slice : X @ W_project -> ascription
Aggregate : ascription.T @ X -> tokens
Attend : tokens間で QK^T attention
Deslice : ascription @ updated_tokens -> X_updated
```
```

ascription は「割り当て」「所属度」という意味です。Transolverの文脈では、点が各sliceへどれくらい属するかを表す重みです。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 22

<!-- codex-code-explanation -->

コード解説: Physics-Attention層としてまとめる

このセルでは、Slice、Aggregate、Token Attention、Desliceの4段階を1つの PhysicsAttention クラスにまとめます。実際のTransolverでは、これに線形層、MLP、正規化、残差接続などが加わり、Transformer blockとして学習されます。ここでは重み更新は行いませんが、forward計算の流れをコードとして確認できます。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Code 6

--- 4つの処理を、1つのPhysics-Attention層としてまとめます。 ---

class PhysicsAttention:

初期化では、slice割り当て用とtoken Attention用の重みを用意します。

def __init__(self, n_slices, d_model, seed=0):

rng = np.random.default_rng(seed)

self.n_slices = n_slices

self.d_model = d_model

self.W_project = rng.random((d_model, n_slices))

self.W_q = rng.random((d_model, d_model))

self.W_k = rng.random((d_model, d_model))

self.W_v = rng.random((d_model, d_model))

forwardでは、入力点群特徴Xを受け取り、更新後の点群特徴を返します。

def forward(self, X):

1. Slice: 各点をsliceへsoft assignment

ascription = softmax(X @ self.W_project, axis=1) # (N, M)

2. Aggregate: 点情報をslice tokenへ集約

denom = ascription.sum(axis=0, keepdims=True).T + 1e-6

tokens = (ascription.T @ X) / denom # (M, C)

3. Attend: token間でAttention

Q = tokens @ self.W_q

K = tokens @ self.W_k

V = tokens @ self.W_v

scores = (Q @ K.T) / np.sqrt(self.d_model)

token_attention = softmax(scores, axis=1)

updated_tokens = token_attention @ V # (M, C)

4. Deslice: token情報を各点へ戻す

```
X_updated = ascription @ updated_tokens    # (N, C)
return X_updated, ascription, token_attention
```

```
# --- 作った層にtoy dataを通し、shapeが保たれることを確認します。 ---
```

```
pa_layer = PhysicsAttention(n_slices=8, d_model=16, seed=42)
```

```
toy_data = np.random.rand(1000, 16)
```

```
output, ascription, token_attention = pa_layer.forward(toy_data)
```

```
print("input shape: ", toy_data.shape)
```

```
print("output shape:", output.shape)
```

```
print("ascription shape:", ascription.shape)
```

```
print("token attention shape:", token_attention.shape)
```

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Code 6 output

input shape: (1000, 16)

output shape: (1000, 16)

ascription shape: (1000, 8)

token attention shape: (8, 8)

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 24

7. 計算量を比較する

標準AttentionとPhysics-Attention風の処理を、主要な行列サイズだけで比較します。

標準Attention:

```
```text
N x N
```
```

Physics-Attention:

```
```text
N x M でslice
M x M でtoken attention
N x M でdeslice
```
```

M が N より十分小さいとき、大きな差になります。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 25

<!-- codex-code-explanation -->

コード解説: 計算量の比較

このセルでは、標準Attentionの N^2 とPhysics-Attentionの概算 $2NM + M^2$ を比較します。N が大き
くても M を64程度に固定できれば、計算量は大幅に小さくなります。CFD surrogateではメッシュ点数
が多いため、この差がGPUメモリと学習時間に直接効きます。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Code 7

```
# --- 標準AttentionとPhysics-Attentionの計算量を関数にします。 ---
def complexity_standard(N):
    return N ** 2

def complexity_physics_attention(N, M):
    return 2 * N * M + M ** 2

# --- 点数Nを増やし、slice数Mを固定したときの差を見ます。 ---
for N_test in [1_000, 10_000, 100_000, 400_000]:
    M_test = 64
    a = complexity_standard(N_test)
    b = complexity_physics_attention(N_test, M_test)
    print(f"N={N_test:>7,}, M={M_test}: standard={a/1e9:>8.2f}B, physics-attn={b/1e6:>8.2f}M, ratio={a/b:>8.1f
}x")
```

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Code 7 output

N= 1,000, M=64: standard= 0.00B, physics-attn= 0.13M, ratio= 7.6x

N= 10,000, M=64: standard= 0.10B, physics-attn= 1.28M, ratio= 77.9x

N=100,000, M=64: standard= 10.00B, physics-attn= 12.80M, ratio= 781.0x

N=400,000, M=64: standard= 160.00B, physics-attn= 51.20M, ratio= 3124.8x

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 27

8. Full Transolver層では何が追加されるか

このNotebookで実装したのはPhysics-Attentionの核だけです。実際のTransolver層では、通常のTransformer blockと同様に、次のような部品が組み合わされます。

- 入力特徴を隠れ次元へ写すMLP
- Physics-Attention
- 残差接続 (Residual connection)
- LayerNorm
- Feed Forward Network / MLP
- 複数層のstack

残差接続 は、層の入力を出力に足す仕組みです。深いネットワークで学習を安定させます。

LayerNorm は、特徴量のスケールを整える正規化です。Transformerでよく使われます。

Notebook 2: Transolver / Physics-AttentionをNumPyで理解する / Markdown 28

まとめ

このNotebookで確認したことは次の通りです。

1. Physics-Attentionは、点を直接すべて比較せず、少数のslice tokenへ集約する。
2. 主要なshapeは $x: (N, C)$, $\text{slice_weights}: (N, M)$, $z: (M, C)$ 。
3. 重いAttentionは $N \times N$ ではなく $M \times M$ に置き換わる。
4. 更新されたtokenは、同じslice重みを使って各点へ戻される。
5. 入力と出力のshapeが同じなので、Transolver層を重ねられる。

次は、PhysicsNeMoなどの実装で、実データをZarrへ変換し、Transolverを学習する流れを確認します。